Trello-Inspired Task Management Backend Schema (FastAPI + SQLAlchemy + Pydantic)

1. User Model (models/user.py)

Defines application users with their credentials and basic profile information. UUID is used as the primary key for scalability and uniqueness.

```
import uuid
from sqlalchemy import Column, String, DateTime
from sqlalchemy.sql import func
from app.core.database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    full_name = Column(String, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
    password_hash = Column(String, nullable=False)
    avatar_url = Column(String, nullable=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())
```

2. Workspace Model (models/workspace.py)

A workspace is a collaborative environment containing multiple boards. Each workspace has members with roles like owner, admin, or member.

```
import uuid
from sqlalchemy import Column, String, DateTime, ForeignKey, Text, Enum
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
import enum
from app.core.database import Base

class WorkspaceRole(str, enum.Enum):
    owner = "owner"
    admin = "admin"
    member = "member"
    guest = "guest"

class Workspace(Base):
    __tablename__ = "workspaces"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    name = Column(String, nullable=False)
    description = Column(Text)
    created_by = Column(String, ForeignKey("users.id"))
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())

    members = relationship("WorkspaceMember", back_populates="workspace")
    boards = relationship("Board", back_populates="workspace")

class WorkspaceMember(Base):
    __tablename__ = "workspace_members"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    workspace_id = Column(String, ForeignKey("workspaces.id"))
    user_id = Column(String, ForeignKey("users.id"))
```

```
    role = Column(Enum(WorkspaceRole), default=WorkspaceRole.member)
    created_at = Column(DateTime(timezone=True), server_default=func.now())

    workspace = relationship("Workspace", back_populates="members")
```

3. Board Model (models/board.py)

A board represents a project within a workspace. It can contain multiple lists and cards. Each board is tied to a workspace and creator.

```
import uuid
from sqlalchemy import Column, String, DateTime, ForeignKey, Text
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
from app.core.database import Base

class Board(Base):
    __tablename__ = "boards"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    workspace_id = Column(String, ForeignKey("workspaces.id"))
    name = Column(String, nullable=False)
    description = Column(Text)
    background_color = Column(String)
    created_by = Column(String, ForeignKey("users.id"))
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())

    workspace = relationship("Workspace", back_populates="boards")
    lists = relationship("List", back_populates="board", cascade="all, delete")
```

4. List Model (models/list.py)

Each board contains multiple lists (columns) such as To Do, In Progress, Done. Lists have position fields for drag-and-drop ordering.

```
import uuid
from sqlalchemy import Column, String, DateTime, ForeignKey, Integer
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
from app.core.database import Base

class List(Base):
    __tablename__ = "lists"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    board_id = Column(String, ForeignKey("boards.id"))
    title = Column(String, nullable=False)
    position = Column(Integer, nullable=False, default=0)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())

    board = relationship("Board", back_populates="lists")
    cards = relationship("Card", back_populates="list", cascade="all, delete")
```

5. Card Model (models/card.py)

Cards represent tasks inside lists. Each card can have a description, due date, priority, and position for ordering. The `Priority` enum improves data consistency.

```
import uuid
import enum
from sqlalchemy import Column, String, DateTime, ForeignKey, Text, Enum, Integer
```

```
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
from app.core.database import Base

class Priority(str, enum.Enum):
    low = "low"
    medium = "medium"
    high = "high"
    urgent = "urgent"

class Card(Base):
    __tablename__ = "cards"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    list_id = Column(String, ForeignKey("lists.id"))
    title = Column(String, nullable=False)
    description = Column(Text)
    due_date = Column(DateTime(timezone=True), nullable=True)
    priority = Column(Enum(Priority), default=Priority.medium)
    created_by = Column(String, ForeignKey("users.id"))
    position = Column(Integer, default=0)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())

    list = relationship("List", back_populates="cards")
```

6. Pydantic Schemas (schemas/*.py)

These Pydantic models define validation and response formats for FastAPI endpoints. They ensure clean data flow between frontend and backend.

```
# schemas/user.py
from pydantic import BaseModel, EmailStr
from typing import Optional
import datetime

class UserBase(BaseModel):
    full_name: str
    email: EmailStr

class UserCreate(UserBase):
    password: str

class UserResponse(UserBase):
    id: str
    avatar_url: Optional[str]
    created_at: datetime.datetime

    class Config:
        orm_mode = True

# schemas/card.py
from pydantic import BaseModel
from typing import Optional
import datetime
from enum import Enum

class Priority(str, Enum):
    low = "low"
    medium = "medium"
    high = "high"
    urgent = "urgent"

class CardBase(BaseModel):
    title: str
    description: Optional[str] = None
```

```
    due_date: Optional[datetime.datetime] = None
    priority: Priority = Priority.medium

class CardCreate(CardBase):
    list_id: str

class CardResponse(CardBase):
    id: str
    created_at: datetime.datetime

    class Config:
        orm_mode = True
```

7. Summary and Best Practices

- **UUIDs** ensure globally unique primary keys. - **Enum fields** help maintain clean and consistent data (e.g., roles, priorities). - **Cascade deletes** ensure relational integrity (delete list → cards also deleted). - **Timestamps** provide auditing capability. - **Separation of models and schemas** keeps logic modular and maintainable. - Ready for extension: add comments, attachments, labels, and activity logs easily.